第 7 章

Linux系统的Python基础

Python语言基础（Python优势）

- **市场需求大，跨领域广泛应用**

Python不仅可以用于传统的软件开发，还广泛应用于科学计算、数据分析、人工智能、机器学习、网络开发、自动化运维等多个领域，掌握Python可以增加就业机会。

- **易于学习，学习资源丰富**

语法简洁易懂，强调代码的可读性和简洁性。因为Python的流行，网上有大量的学习资源，包括免费的教程、视频和在线课程。

- **强大的库支持**

Python拥有丰富的库和框架，如NumPy、Pandas用于数据分析，TensorFlow和PyTorch用于深度学习等。

- **社区支持强大**

Python有一个活跃的社区，遇到问题需要帮助或者想要获取最新的开发资源，都能在社区中找到支持。

- **支持多种编程范式**

Python支持过程式、面向对象和函数式编程，可以根据不同的需求选择最适合的编程范式。

学习目标

- 熟练掌握Python扩展库的安装方法
- 熟练掌握导入与使用标准库、扩展库对象的方法
- 理解Python编码规范
- 了解常用数据类型
- 熟练掌握列表、元组、字典、集合、字符串概念与常用方法
- 熟练掌握常用内置函数的语法和功能
- 熟练掌握常用运算符语法和功能
- 理解选择结构、循环结构、异常处理结构的语法和执行流程

学习目标

- 掌握函数定义与使用的语法
- 理解函数参数与变量作用域的概念
- 掌握定义与使用类的语法
- 理解数据成员、成员方法、属性的概念
- 理解私有成员与公有成员的概念

Python开发环境搭建与使用

jupyter

```
In [1]:
1 3+5
```

```
Out[1]: 8
```

```
In [2]:
1 print(3+5)
```

```
8
```

```
In [7]:
1 import math
2 math.comb(5,3)  #从5个数中选取3个数的组合数
```

```
Out[7]: 10
```

```
In [6]:
1 print(math.gcd(72,96))   #计算输出最大公约数
```

```
24
```

```
In [8]:
1 from random import sample
2 print(sample(range(10),5))  #随机选取5个小于10的不重复整数。以列表形式返回
```

```
[2, 1, 9, 6, 5]
```

Python开发环境搭建与使用

jupyter

In [4]:

```
import time
import pandas as pd
from pyecharts.charts import Bar, Line, Pie
from pyecharts import options as opts
from pyecharts.globals import SymbolType, ThemeType


# 读取lagou01.csv文件的数据
recruit_obj = pd.read_csv('lagou01.csv', encoding='gbk')
# 过滤与分析目标无关的数据，保留有关的数据
new_df_01 = pd.DataFrame([recruit_obj['city'],
                          recruit_obj['companyFullName'], recruit_obj['salary'],
                          recruit_obj['companySize'], recruit_obj['district'],
                          recruit_obj['education'], recruit_obj['firstType'],
                          recruit_obj['positionAdvantage'], recruit_obj['workYear'],
                          recruit_obj['createTime']]).T
new_df_01.head(5)
```

Out[4]:

	city	companyFullName	salary	companySize	district	education	firstType	positionAdvantage	workYear	createTime
0	北京	达疆网络科技（上海）有限公司	15k-30k	2000人以上	大兴区	本科	产品|需求|项目类	成长快、氛围好、领导好	3-5年	2019/12/2 20:38
1	北京	北京音娱时光科技有限公司	10k-18k	50-150人	海淀区	本科	产品|需求|项目类	技术大牛多；免费餐饮；氛围好；	1-3年	2019/12/3 11:23
2	北京	北京千喜鹤餐饮管理有限公司	20k-30k	2000人以上	海淀区	本科	产品|需求|项目类	福利好，五险一金，住房补助	3-5年	2019/12/3 10:35
3	北京	吉林省海生电子商务有限公司	33k-50k	少于15人	朝阳区	本科	产品|需求|项目类	五险一金	3-5年	2019/12/3 10:35
4	北京	韦博网讯科技（北京）有限公司	10k-15k	50-150人	朝阳区	本科	产品|需求|项目类	待遇优厚 ，良好的发展前景	1-3年	2019/12/3 12:10

Python开发环境搭建与使用

西北民族大學 NORTHWEST MINZU UNIVERSITY

jupyter

In [44]:
```
from pyecharts.globals import WarningType
WarningType.ShowWarning = False

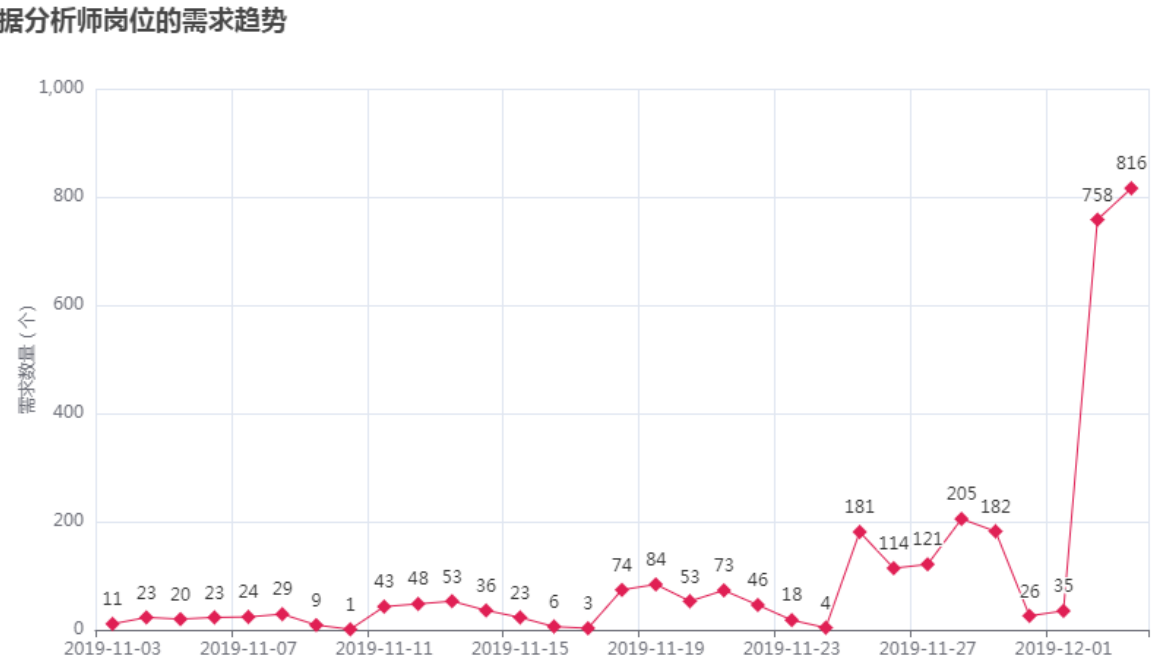
#构建图例
line_demo = (
    Line(init_opts=opts.InitOpts(theme=ThemeType.ROMA))

    # 添加x轴、y轴的数据、系列名称
    .add_xaxis(jobs_count.index.tolist())
    .add_yaxis('', jobs_count.values.tolist(), symbol='diamond', symbol_size=10)

    # 设置标题
    .set_global_opts(title_opts = opts.TitleOpts(title="数据分析师岗位的需求趋势"),
                     yaxis_opts = opts.AxisOpts(name="需求数量（个）", name_location="center", name_gap=40))
)

#渲染图例
line_demo.render_notebook()
```

Out[44]:



Python开发环境搭建与使用

jupyter

In [11]:
```
# 从标准库random中导入函数shuffle()，导入时不要加括号
from random import shuffle

# 创建列表，range(10)  等价于  range(0,10,1)
# 返回左闭右开区间[0,10)中以1为步长的整数组成的范围对象

data = list(range(10))
#输出列表的值
print(data)
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

In [12]:
```
# 随机打乱列表中元素的顺序
shuffle(data)
print(data)
```

```
[4, 6, 1, 3, 0, 7, 5, 8, 2, 9]
```

In [13]:
```
# 调用列表的sort()方法，把列表中的元素从小到大升序排序
data.sort()
print(data)
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

安装扩展库

pip子命令示例	说明
pip freeze [>requirements.txt]	列出已安装扩展库及其版本号，可以使用重定向功能把信息写入文本文件
pip install SomePackage[==version]	在线安装扩展库，必要时可以指定版本，不指定时默认安装最新版本，使用时把SomePackage替换为实际的扩展名名称，例如jieba、pillow、pypinyin
pip install -r requirements.txt	从文本文件（由当前表格第一行的freeze子命令创建）中读取扩展库信息并安装扩展库
pip install SomePackage.whl	通过whl文件离线安装扩展库
pip install --upgrade SomePackage	升级SomePackage模块到最新版本
pip uninstall SomePackage	卸载SomePackage模块

安装扩展库

西北民族大學 NORTHWEST MINZU UNIVERSITY

jupyter

```
1 !pip freeze
```

```
json5 @ file:///tmp/build/80754af9/json5_1624432770122/work
jsonschema @ file:///opt/concourse/worker/volumes/live/4dfc22c3-c342-49fd-67d7-73b2208a33ed
/volume/jsonschema_1650025962276/work
jupyter @ file:///opt/concourse/worker/volumes/live/5d55c245-fcac-42f9-4a6c-7e147c07785b/vo
lume/jupyter_1607700866889/work
jupyter-client @ file:///tmp/build/80754af9/jupyter_client_1616770841739/work
jupyter-console @ file:///tmp/build/80754af9/jupyter_console_1616615302928/work
jupyter-core @ file:///opt/concourse/worker/volumes/live/0be57320-7276-4b32-7428-a5506d17e8
62/volume/jupyter_core_1646994454283/work
jupyter-server @ file:///opt/conda/conda-bld/jupyter_server_1644494914632/work
jupyterlab @ file:///opt/conda/conda-bld/jupyterlab_1647445413472/work
jupyterlab-pygments @ file:///tmp/build/80754af9/jupyterlab_pygments_1601490720602/work
jupyterlab-server @ file:///opt/conda/conda-bld/jupyterlab_server_1644500396812/work
jupyterlab-widgets @ file:///tmp/build/80754af9/jupyterlab_widgets_1609884341231/work
keyring @ file:///opt/concourse/worker/volumes/live/33b3e085-74e6-4f7b-77a0-470eb43e69ae/vo
lume/keyring_1638531366470/work
kiwisolver @ file:///Users/builder/miniconda3/envs/prefect/conda-bld/kiwisolver_16385698857
39/work
```

安装扩展库

西北民族大學 NORTHWEST MINZU UNIVERSITY

jupyter

In [20]:
```
1 !pip install jieba
```

```
Looking in indexes: https://pypi.tuna.tsinghua.edu.cn/simple
Collecting jieba
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/c6/cb/18eeb235f833b726522d7ebed54f2278ce28ba9438e3135ab0278d9792a2/jieba-0.42.1.tar.gz (19.2 MB)
     |████████████████████████████████| 19.2 MB 1.6 MB/s eta 0:00:01
Building wheels for collected packages: jieba
  Building wheel for jieba (setup.py) ... done
  Created wheel for jieba: filename=jieba-0.42.1-py3-none-any.whl size=19314478 sha256=722e2b910690d5dc12e9fe47763e9a7d2d662aff7a3d23d45eb9ac3e4df2c685
  Stored in directory: /Users/xianghe/Library/Caches/pip/wheels/1a/76/68/b6d79c4db704bb18d54f6a73ab551185f4711f9730c0c15d97
Successfully built jieba
Installing collected packages: jieba
Successfully installed jieba-0.42.1
```

In [22]:
```
1 !pip install jieba==0.42.1
```

```
Looking in indexes: https://pypi.tuna.tsinghua.edu.cn/simple
Requirement already satisfied: jieba==0.42.1 in /Users/xianghe/Documents/anaconda3/lib/python3.9/site-packages (0.42.1)
```

安装扩展库

jupyter

jieba 是一个中文分词Python库。以下是一个简单的例子，展示如何使用 jieba 进行中文分词。

```
1  import jieba
2
3  # 示例文本
4  text = "我是西北民族大学计算机科学与技术专业的本科生"
5
6  # 使用精确模式进行分词
7  seg_list = jieba.cut(text, cut_all=False)
8  print("精确模式: " + "/ ".join(seg_list))
9
10 # 使用全模式进行分词
11 seg_list = jieba.cut(text, cut_all=True)
12 print("全模式: " + "/ ".join(seg_list))
13
14 # 使用搜索引擎模式进行分词
15 seg_list = jieba.cut_for_search(text)
16 print("搜索引擎模式: " + "/ ".join(seg_list))
```

```
精确模式: 我/ 是/ 西北/ 民族/ 大学/ 计算机科学/ 与/ 技术/ 专业/ 的/ 本科生
全模式: 我/ 是/ 西北/ 民族/ 大学/ 计算/ 计算机/ 计算机科学/ 算机/ 科学/ 与/ 技术/ 专业/ 的/ 本科/ 本科生
搜索引擎模式: 我/ 是/ 西北/ 民族/ 大学/ 计算/ 算机/ 科学/ 计算机/ 计算机科学/ 与/ 技术/ 专业/ 的/ 本科/ 本科生
```

安装扩展库

- 由于网速问题导致在线安装速度过慢，pip命令支持指定国内的站点来提高速度，下面的命令用来从阿里云服务器下载安装扩展库jieba，其他服务器地址可以自行查阅。

  pip install 包名(jieba) -i http://mirrors.aliyun.com/pypi/simple

  pip install 包名(jieba) -i https://pypi.tuna.tsinghua.edu.cn/simple

- 上面的方法为临时使用，在安装某个包时，可以在pip命令中直接指定镜像源。要永久设置pip使用某个国内镜像源，您可以修改pip的配置文件。以下是具体步骤：

  pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple

```
1 !pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple
Writing to /Users/xianghe/.config/pip/pip.conf
```

安装扩展库

- **如果扩展库暂时不支持自己使用的Python版本或者需要**

➢可以从指定网站中下载第三方编译好的whl文件（.whl格式的扩展库二进制安装文件），然后离线安装。要注意：一定要选择正确版本（文件名中有cp38表示适用于Python 3.8，有cp37表示适用于Python 3.7；以此类推；文件名中有win32表示适用于32位系统，有win_amd64表示适用于64位系统），一定不要修改下载的文件名。然后使用pip命令进行离线安装，指定文件的完整路径和扩展名。命令如下：

```
pip install psutil-5.6.7-cp38-cp38-win_amd64.whl
```

1.1导入与使用标准库、扩展库对象

（1）使用“import 模块名[as 别名]”的方式将**模块导入**，使用其中的对象时需要在对象之前加上模块名作为前缀，以“**模块名.对象名**”的形式进行访问。

如果模块名字很长的话，可以为导入的模块设置一个别名，然后使用“**别名.对象名**”的方式来使用其中的对象。

```
1  import jieba
2
3  # 示例文本
4  text = "我是西北民族大学计算机科学与技术专业的本科生"
5
6  # 使用精确模式进行分词
7  seg_list = jieba.cut(text, cut_all=False)
8  print("精确模式: " + "/ ".join(seg_list))
```

1.1 导入与使用标准库、扩展库对象

（2）使用“from 模块名/库名 import 对象名/模块名[as 别名]”的方式仅**导入明确指定的对象**，使用对象时不需要使用模块名作为前缀，可以减少程序员需要输入的代码量。使用这种形式既可以导入库中的模块，也可以导入模块中的对象。

```
1 from random import sample
2 print(sample(range(100), 10))    # 从[0,100)区间选择10个不重复
```

```
[83, 0, 58, 52, 25, 60, 70, 88, 51, 91]
```

1.1 导入与使用标准库、扩展库对象

（3）使用“from 模块名 import *”的方式**可以一次导入模块中的所有对象**或者特殊成员__all__列表中明确指定的对象，使用模块中的对象时不需要使用模块名作为前缀。**一般并不建议这样使用**，除非是程序中用到了某个库中的大部分对象。

例：查看当前的日期时间。

```
1 from time import *
2 print(localtime())
```

```
time.struct_time(tm_year=2024, tm_mon=8, tm_mday=28, tm_hour=17, tm
_min=52, tm_sec=24, tm_wday=2, tm_yday=241, tm_isdst=0)
```

1.2 Python编码规范

➢ 缩进

- **在函数定义、类定义、选择结构、循环结构、异常处理结构和with语句等结构中，对应的函数体或语句块都必须有相应的缩进。**当某一行代码与上一行代码不在同样的缩进层次上，并且与之前某行代码的缩进层次相同，表示上一行代码所在的语句块结束。

具有相同缩进量的连续多行代码是顺序结构，从上往下依次执行每行代码。

```
1 for i in list(range(3)):
2     print(i)
3     print(i+1)
4     print("--")
5 print("end")
```

```
0
1
--
1
2
--
2
3
--
end
```

1.2 Python编码规范

➢ 续行符

- 尽量不要写过长的语句，应尽量保证一行代码不超过屏幕宽度，以提高可读性。如果语句确实太长超过屏幕宽度，（1）在行尾使用续行符“\”表示下一行代码仍属于本条语句，（2）使用圆括号把多行代码括起来表示是一条语句，这两种形式属于显式续行；（3）列表、元组、字典、集合中的元素过多时可以直接在某个元素后逗号后面进行换行，这属于隐式续行的用法。

```
expression1 = 1 + 2 + 3\      # 使用反斜线作为续行符
              + 4 + 5
expression2 = (1 + 2 + 3      # 把多行表达式放在圆括号中表示是一条语句
               + 4 + 5)
data = [80, 90, 100,          # 隐式续行
        60, 70, 80]
```

1.2 Python编码规范

➢ 注释

在Python中有两种常用的注释形式：**井号“#”**和**三引号**。

1. 井号“#”用于单行注释，表示本行中“#”符号之后的内容不作为代码运行，一般建议在表示注释的井号“#”后面增加一个空格再写注释内容，并且同一段代码中行内注释的井号尽量垂直对齐；
2. 一对三引号（三单引号或三双引号都可以）常用于大段说明性文本的注释，也可以用于表示包含多行的字符串。（记得要成对出现，不能出现：前面是三个单引号开始，后面用三个双引号结束）。

1.3 常用数据类型

- 一个对象所属的数据类型决定了这个对象的取值范围和支持的操作。
- 在Python语言中所有的一切都可以称作对象，常见的对象类型有整数、实数、复数、字符串、列表、元组、字典、集合和zip对象、map对象、enumerate对象、range对象、filter对象、生成器对象等内置对象，以及大量标准库对象和扩展库对象，函数和类也可以称作对象。

1.3 常用数据类型

西北民族大學 NORTHWEST MINZU UNIVERSITY

Python常用内置类型

对象类型	类型名称	示例	简要说明
数字	int float complex（复数）	666，0o777，0b1111，0x4ed8 2.718281828459045，1.2e-3 3+4j，5J	数字大小没有限制，0o开头表示八进制数、0b开头表示二进制数、0x开头表示十六进制数；1.2e-3表示1.2乘以10的-3次方；复数中j或J表示虚部
字符串	str	'Readability counts.' "What's your name?" '"Tom said, "let's go."'" r'C:\Windows\notepad.exe' f'My name is {name}' rf'{directory}\{fn}'"	使用单引号、双引号、三引号作为定界符，不同定界符之间可以互相嵌套，三引号内可以包含多行字符串；引号前面加字母r或R表示原始字符串，任何字符都不进行转义；前面加字母f或F表示对字符串中大括号内的变量名占位符进行替换，对字符串进行格式化；

```
# result = 5 / 2 # 结果为2.5
#result = 5 // 2 # 结果为2
#remainder = 5 % 2 # 结果为1
```

```
1 name = "LiMing"
2 age = 30
3
4 # 使用f-string进行格式化
5 formatted_string = f"My name is {name} and I am {age} years old."
6
7 print(formatted_string)
8
```

```
My name is LiMing and I am 30 years old.
```

1.3 常用数据类型

对象类型	类型名称	示例	简要说明
字节串	bytes	b'\xe8\xae\xa1\xe7\xae\x97\xe6\x9c\xba'	以字母b引导，可以使用单引号、双引号、三引号作为定界符，可以使用字节串方法decode() 同一个字符串使用不同编码格式编码得到的字节串可能会不一样，字节串解码为字符串时也需要指定相应的解码格式。
列表	list	['red','green','blue'] ['a', {3}, (1,2), ['c', 2], {65:'A'}] []	➢ 所有元素放在一对方括号中，元素之间使用逗号分隔 ➢ 元素可以是任意类型 ➢ 一对空的方括号表示空列表

```python
# 定义解码后的字符串
decoded_string = "计算机"

# 使用UTF-8编码将字符串编码为字节字符串
encoded_byte_string = decoded_string.encode('utf-8')

# 打印编码后的字节字符串
print(encoded_byte_string)
```

```
b'\xe8\xae\xa1\xe7\xae\x97\xe6\x9c\xba'
```

```python
# 定义字节字符串
byte_string = b'\xe8\xae\xa1\xe7\xae\x97\xe6\x9c\xba'

# 使用UTF-8编码解码字节字符串
decoded_string = byte_string.decode('utf-8')

# 打印解码后的字符串
print(decoded_string)
```

```
计算机
```

1.3 常用数据类型

对象类型	类型名称	示例	简要说明
元组	tuple	(0, 0, 255) (0,) ()	➢ 所有元素放在一对圆括号中，元素之间使用逗号分隔， ➢ 元组中只有一个元素时后面的逗号不能省略， ➢ 一对空的圆括号表示空元组
字典	dict	{'red': (1,0,0), 'green': (0,1,0), 'blue': (0,0,1)} {}	➢ 所有元素放在一对大括号中，元素之间使用逗号分隔，元素形式为“键:值”； ➢ “键”不允许重复并且必须为不可变类型，“值”可以是任意类型的数据 ➢ 一对空的大括号表示空字典
集合	set	{'red', 'green', 'blue'} set()	➢ 所有元素放在一对大括号中，元素之间使用逗号分隔； ➢ 元素不允许重复且必须为不可变类型的对象。 ➢ set()表示空集合，不能使用一对空的大括号{}表示空集合

1.3 常用数据类型

对象类型	类型名称	示例	简要说明
布尔型	bool	True, False	逻辑值，**首字母必须大写**
空类型	NoneType	None	空值，**首字母必须大写**
异常	NameError ValueError TypeError KeyError.....		Python内置异常类
文件		fp = open('data.txt', 'w', encoding='utf8')	Python内置函数**open()**使用指定的模式打开文件，返回文件对象
迭代器		生成器对象、zip对象、enumerate对象、map对象、filter对象、reversed对象等等	空间占用小，适合大数据处理；**只能从前向后逐个访问其中的元素，并且每个元素只能使用一次**

1.3 常用数据类型

迭代器使用

```python
# 显式地使用迭代器
my_iter = iter([1, 2, 3, 4, 5])  # 创建一个迭代器
print(next(my_iter))  # 输出 1
print(next(my_iter))  # 输出 2
```

```
1
2
```

```python
# 定义两个列表
names = ['Alice', 'Bob', 'Charlie']
ages = [25, 30, 35]

# 使用zip创建一个迭代器
zipper = zip(names, ages)

# 遍历zip对象
for name, age in zipper:
    print(f'{name} is {age} years old.')
```

```
Alice is 25 years old.
Bob is 30 years old.
Charlie is 35 years old.
```

```python
# 定义两个列表
names = ['Alice', 'Bob', 'Charlie']
ages = [25, 30, 35]

# 使用zip创建一个迭代器
zipper = zip(names, ages)
print(next(zipper))
print(next(zipper))
print(next(zipper))
```

```
('Alice', 25)
('Bob', 30)
('Charlie', 35)
```

1.3 常用数据类型

- 在Python中，不需要事先声明变量名及其类型，使用赋值语句可以直接创建任意类型的变量，**变量的类型取决于等号右侧表达式计算结果的类型**。
- 赋值语句的执行过程是：
  - 首先把等号右侧表达式的值计算出来，
  - 然后在内存中寻找一块合适的空间把值**存放进去**，
  - 最后创建变量并**指向或引用**这个内存地址（贴上一个标签）。对于不再使用的变量，可以使用del语句将其删除（可以理解为撕掉标签）。

1.3 常用数据类型


```
>>> data = 3 + 5                                # 创建整型变量
>>> type(data)                                  # 查看变量类型
<class 'int'>
>>> print(data*3)                               # 使用变量的值
24
>>> data = 3.14                                 # 创建实型变量
>>> type(data)
<class 'float'>
>>> del data
>>> data                                        # 变量已被删除，再访问会出错
Traceback (most recent call last):
  File "<pyshell#31>", line 1, in <module>
    data
NameError: name 'data' is not defined
```

1.3.1 整数、实数、复数


```
1 import math
2 print(math.factorial(20))  # 计算20的阶乘
```

```
2432902008176640000
```

```
1 print(0.4 - 0.3)           # 实数运算可能会有误差
```

```
0.10000000000000003
```

```
1 print(0.4-0.3 == 0.1)      # 直接比较实数是否相等，可能得到错误结果
```

```
False
```

```
1 print(math.isclose(0.4-0.3, 0.1)) # 测试两个实数是否足够接近
```

```
True
```

```
1 c = 3 + 4j
2 print(abs(c))                    # 计算复数的模
```

```
5.0
```

1.3.1 整数、实数、复数

在Python中，有一个**Decimal数据类型**，用于十进制浮点运算。与Python内置的浮点数类型float相比，优点：

- **精度控制**：可以设置任意精度的算术运算。
- **可预测性**：Decimal对象的行为更加可预测，这对于金融和货币计算尤其重要。
- **可定制性**：可以配置Decimal对象的行为，例如设置舍入模式。

```
1 import decimal
2 value = decimal.Decimal('3.14')          # 创建精确的高精度实数
3 value
```

```
Decimal('3.14')
```

```
1 value * 3
```

```
Decimal('9.42')
```

```
1 decimal.Decimal(1/3)  # 使用实数创建，有误差，不建议使用实数来创建。
```

```
Decimal('0.33333333333333331482961625624739099293947219848632812 5')
```

```
1 decimal.Decimal(1) / decimal.Decimal(3)  # 高精度实数除法
```

```
Decimal('0.33333333333333333333333333')
```

```
1 decimal.getcontext()      # 查看当前上下文
```

```
Context(prec=28, rounding=ROUND_HALF_EVEN, Emin=-999999, Emax=999999, c
apitals=1, clamp=0, flags=[Inexact, FloatOperation, Rounded], traps=[In
validOperation, DivisionByZero, Overflow])
```

```
1 decimal.getcontext().prec = 100          # 设置小数位数
```

```
1 decimal.Decimal(1) / decimal.Decimal(3)
```

```
Decimal('0.3333333333333333333333333333333333333333333333333333333333
3333333333333333333333333333333333333')
```

1.3.2 列表、元组、字典、集合

	列表	元组	字典	集合
类型名称	list	tuple	dict	set
定界符	方括号[]	圆括号()	大括号{}	大括号{}
是否可变	是	否	是	是
是否有序	是	是	否	否
是否支持下标	使用序号作下标	使用序号作下标	使用“键”作下标	否
元素分隔符	逗号	逗号	逗号	逗号
对元素形式的要求	无	无	键:值	必须可哈希
对元素值的要求	无	无	“键”必须可哈希	必须可哈希
元素是否可重复	是	是	“键”不允许重复，“值”可以重复	否
元素查找速度	很慢	很慢	非常快	非常快
新增和删除元素速度	尾部操作快，其他位置慢	不允许	快	快

1.3.2 列表常用方法

列表对象常用方法（1）

方法	功能描述
append(object)	将object追加至当前列表的尾部，不影响列表中已有的元素下标，也不影响列表在内存中的起始地址，没有返回值。
count(value)	返回value在当前列表中的出现次数
extend(iterable)	将可迭代对象iterable中所有元素追加至当前列表的尾部
insert(index, object)	在当前列表的index位置前面插入对象object，该位置及后面所有元素自动向后移动，索引加1
index(value, start=0, stop=1024)	返回当前列表指定范围中第一个值为value的元素的索引，若不存在值为value的元素则抛出异常ValueError，可以使用参数start和stop指定要搜索的下标范围，start默认为0表示从头开始，stop默认值为最大允许的下标值

1.3.2 列表常用方法

列表对象常用方法（2）

pop(index=-1)	删除并返回当前列表中下标为index的元素，该位置后面的所有元素自动向前移动，索引减1。index默认为-1，表示删除并返回列表中最后一个元素。
remove(value)	在当前列表中删除第一个值为value的元素，被删除元素所在位置之后的所有元素自动向前移动，索引减1。
reverse()	对当前列表中的所有元素进行原地翻转，首尾交换
sort(key=None, reverse=False)	对当前列表中的元素进行原地排序，是稳定排序（在指定规则下相等的元素保持原来的相对顺序）。参数key用来指定排序规则，可以为任意可调用对象；参数reverse为False表示升序，True表示降序。

1.3.2 字典常用方法

字典对象常用方法（1）

方法	功能描述
clear()	不接收参数，删除当前字典对象中的所有元素
get(key, default=None)	返回当前字典对象中以参数key为“键”对应的元素的“值”
items()	不接收参数，返回包含当前字典对象中所有元素的dict_items对象，其中每个元素形式为元组(key,value)

1.3.2 字典常用方法

字典对象常用方法（2）

方法	功能描述
keys()	不接收参数，**返回当前字典对象中所有的“键”**，结果为dict_keys类型的可迭代对象，可以直接和集合进行并集、交集、差集等运算
pop(k[,d])	**删除以k为“键”的元素，返回对应元素的“值”**，如果当前字典中没有以k为“键”的元素，返回参数d，此时如果没有指定参数d，抛出KeyError异常
popitem()	不接收参数，删除并按后进先出顺序返回一个元组(key, value)。
values()	不接收参数，返回包含当前字典对象中所有的“值”的dict_values对象，不能和集合之间进行任何运算

1.3.2 集合常用方法

Python集合对象常用方法（1）

方法	功能描述
add(...)	**往当前集合中增加一个可哈希元素**，如果集合中已经存在该元素，直接忽略该操作，如果参数不可哈希，抛出TypeError异常并提示参数不可哈希。该方法直接修改当前集合，没有返回值
clear()	**删除当前集合对象中所有元素**，没有返回值
difference(...)	接收一个或多个集合（或其他可迭代对象），返回当前集合对象与所有参数对象的差集，不对当前集合做任何修改，功能类似于差集运算符“-”
difference_update(...)	接收一个或多个集合（或其他可迭代对象），**从当前集合中删除所有参数对象中的元素，对当前集合进行更新**，该方法没有返回值，功能类似于运算符“-=”
discard(...)	接收一个可哈希对象作为参数，从当前集合中删除该元素，如果参数元素不在当前集合中则直接忽略该操作。该方法直接修改当前集合，没有返回值

1.3.2 集合常用方法


```python
# 定义两个集合
set_a = {1, 2, 3, 4, 5}
set_b = {4, 5, 6, 7, 8}

# 使用difference方法找出set_a中有而set_b中没有的元素
difference_set = set_a.difference(set_b)

print(difference_set)
```

```
{1, 2, 3}
```

```python
# 定义两个集合
set_a = {1, 2, 3, 4, 5}
set_b = {4, 5, 6, 7, 8}

# 使用difference_update方法更新set_a
set_a.difference_update(set_b)
print(set_a)
```

```
{1, 2, 3}
```

1.3.2 集合常用方法

Python集合对象常用方法（2）

方法	功能描述
intersection(...)	接收一个或多个集合对象，返回当前集合与所有参数对象的交集，不对当前集合做任何修改，类似于运算符“&”
intersection_update(...)	接收一个或多个集合，使用当前集合与所有参数对象的交集更新当前集合对象，没有返回值，类似于运算符“&=”
pop()	不接收参数，**删除并返回当前集合中的任意一个元素**，如果当前集合为空则抛出KeyError异常
remove(...)	**从当前集合中删除参数指定的元素**，如果参数指定的元素不在集合中就抛出KeyError异常，该方法直接修改当前集合，没有返回值
union(...)	接收一个或多个集合，返回当前集合与所有参数对象的并集，不对当前集合做任何修改，类似于并集运算符“|”
update(...)	接收一个或多个集合，把参数对象中所有元素添加到当前集合对象中，没有返回值，功能类似于运算符“|=”

1.3.2 集合常用方法


```python
# 定义三个集合
set_a = {1, 2, 3, 4, 5}
set_b = {4, 5, 6, 7, 8}
set_c = {4, 9, 10}

# 使用intersection方法找出set_a, set_b和set_c的交集
common_elements = set_a.intersection(set_b, set_c)

print(common_elements)
```

```
{4}
```

```python
# 定义三个集合
set_a = {1, 2, 3, 4, 5}
set_b = {4, 5, 6, 7, 8}
set_c = {4, 9, 10}

# 使用intersection_update方法更新set_a为set_a, set_b和set_c的交集
set_a.intersection_update(set_b, set_c)

print(set_a)  # 输出 {4}
```

```
{4}
```

1.3.3 字符串

字符串是包含若干字符的容器对象，其中可以包含汉字、英文字母、数字和标点符号等任意字符。

字符串使用单引号、双引号、三单引号或三双引号作为界定符，**其中三引号里的字符串可以换行**，并且不同的界定符之间可以互相嵌套。

如果字符串中含有反斜线“\”，反斜线和后面紧邻的字符可能会组合成转义字符，这样的组合就会变成其他的含义而不再表示原来的字面意思，例如'\n'表示换行符，'\r'表示回车键，'\b'表示退格键，'\f'表示换页符，'\t'表示水平制表符。

1.3.3 字符串

如果不想反斜线和后面紧邻的字符组合成为转义字符，可以在字符串前面直接加上字母r或R使用原始字符串。

在字符串前面加字母f或F表示对字符串进行格式化（俗称f-字符串），把其中的变量名占位符替换为具体的值。

原始字符串和f-字符串可以同时使用，也就是在字符串前面可以同时加字母f和r（不区分大小写）。

1.3.3 字符串

Python字符串对象常用方法（1）

方法	功能描述
center(width, fillchar=' ') ljust(width, fillchar=' ') rjust(width, fillchar=' ')	返回指定长度的新字符串，当前字符串在新字符串中居中/居左/居右。
count(sub[, start[, end]])	返回字符串sub在当前字符串下标范围[start,end)内不重叠出现的次数，参数start默认值为0，参数end默认值为字符串长度。例如，'abababab'.count('aba')的值为2
encode(encoding='utf-8', errors='strict')	返回当前字符串使用参数encoding指定的编码格式编码后的字节串。对于非ASCII字符，UTF-8编码得到的字节串较长，网络传输时占用带宽较大，保存为文件时占用空间较大；GBK编码得到的字节串更短一些，但有些字符不在GBK字符集中，无法使用GBK编码

1.3.3


```python
text = "Hello"

# 将字符串居中，宽度为10，默认填充字符为空格
centered_text = text.center(10)
print(f'"{centered_text}"')  # 输出 "  Hello   "

# 将字符串左对齐，宽度为10，默认填充字符为空格
left_justified_text = text.ljust(10)
print(f'"{left_justified_text}"')  # 输出 "Hello     "

# 将字符串右对齐，宽度为10，默认填充字符为空格
right_justified_text = text.rjust(10)
print(f'"{right_justified_text}"')  # 输出 "     Hello"
```

```
"  Hello   "
"Hello     "
"     Hello"
```

```python
text = "Hello"

# 将字符串居中，宽度为10，默认填充字符为空格
centered_text = text.center(4)
print(f'"{centered_text}"')  # 输出 "  Hello   "

# 将字符串左对齐，宽度为10，默认填充字符为空格
left_justified_text = text.ljust(4)
print(f'"{left_justified_text}"')  # 输出 "Hello     "

# 将字符串右对齐，宽度为10，默认填充字符为空格
right_justified_text = text.rjust(4)
print(f'"{right_justified_text}"')  # 输出 "     Hello"
```

```
"Hello"
"Hello"
"Hello"
```

1.3.3 字符串

Python字符串对象常用方法（2）

方法	功能描述
endswith(suffix[, start[, end]]) startswith(prefix[, start[, end]])	如果当前字符串下标范围[start, end)的子串以某个字符串suffix/prefix或元组suffix/prefix**指定的几个字符串之一结束/开始则返回True，否则返回False**
find(sub[, start[, end]]) rfind(sub[, start[, end]])	返回字符串sub在当前字符串下标范围[start, end)内**出现**的最小/最大下标，不存在时返回-1
format(*args, **kwargs)	返回对当前字符串进行格式化后的新字符串，其中args表示位置参数，kwargs表示关键参数。
index(sub[, start[, end]]) rindex(sub[, start[, end]])	返回字符串sub在当前字符串下标范围[start, end)内**出现**的最小/最大下标，不存在时抛出ValueError异常并提示子串不存在

1.3.3 字符串

1. find()方法

- 如果子字符串存在，find()返回子字符串首次出现的位置索引。
- 如果子字符串不存在，find()返回-1。

2. index()方法

- 如果子字符串存在，index()返回子字符串首次出现的位置索引。
- 如果子字符串不存在，index()会引发一个ValueError异常。

1. rfind()方法

- 如果子字符串存在，rfind()返回子字符串最后一次出现的位置索引。
- 如果子字符串不存在，rfind()返回-1。

2. rindex()方法

- 如果子字符串存在，rindex()返回子字符串最后一次出现的位置索引。
- 如果子字符串不存在，rindex()会引发一个ValueError异常。

1.3.3 字符串

Python字符串对象常用方法（3）

方法	功能描述
join(iterable)	**使用当前字符串作为连接符把参数iterable中的所有字符串连接成为一个长字符串并返回连接之后的长字符串**，要求参数iterable指定的可迭代对象中所有元素全部为字符串
lower() upper()	返回当前字符串中所有字母都变为**小写/大写**之后的新字符串，非字母字符保持不变
lstrip(chars=None) rstrip(chars=None) **strip**(chars=None)	返回当前字符串删除左侧/右侧/两侧的空白字符或参数chars中所有字符之后的新字符串

1.3.3 字符串

Python字符串对象常用方法（4）

方法	功能描述
replace(old, new, count=-1)	返回当前字符串中所有子串old都被替换为子串new之后的新字符串，参数count用来指定最大替换次数，-1表示全部替换
rsplit(sep=None, maxsplit=-1) split(sep=None, maxsplit=-1)	使用参数sep指定的字符串对当前字符串从后向前/从前向后进行切分，返回包含切分后所有子串的列表。
translate(table)	根据参数table指定的映射表对当前字符串中的字符进行替换并返回替换后的新字符串，不影响原字符串，参数table一般为字符串方法maketrans()创建的映射表，其中映射为空值None的字符将会被删除而不出现在新字符串中

1.3.3 字符串


```
intab = "a"   # 字符串中要替换的字符
outtab = "A"  # 替换成的字符
trantab = str.maketrans(intab, outtab)

original_text = "apple and banana"  # 原始字符串

translated_text = original_text.translate(trantab) # 使用translate()进行字符替换

print(translated_text)  # 输出 "Apple And bAnAnA"
```

```
Apple And bAnAnA
```

```
# 创建一个转换表，将 'a' 替换为 'A' 并删除 'e'
trantab = str.maketrans('a', 'A', 'e')

original_text = "aardvark and elephant"   # 原始字符串

translated_text = original_text.translate(trantab)# 使用translate()进行替换和删除

print(translated_text)  # 输出 "Ardvark And phAntl"
```

```
AArdvArk And lphAnt
```

1.4 运算符语法与功能

- 虽然Python运算符有一套严格的优先级规则（precedence），但并不建议编写程序时过于依赖运算符自身的优先级和结合性，而是应该在编写复杂表达式时使用圆括号来明确其中的逻辑和计算顺序来提高代码可读性。

1.4 运算符语法与功能

Python运算符（1）

运算符	功能说明
lambda [parameter]: expression	用来定义lambda表达式，功能相当于函数，parameter相当于函数参数，可以没有，如果有多个的话使用逗号分隔；expression表达式的值相当于函数返回值
value1 if condition else value2	用来表示一个二选一的表达式，其中value1、condition、value2都为表达式，如果condition的值等价于True则整个表达式的值为value1的值，否则整个表达式的值为value2的值
or	“逻辑或”运算，表示“或者”的意思，具有惰性求值特点。以exp1 or exp2为例，如果exp1的值等价于True则返回exp1的值，否则返回exp2的值
and	“逻辑与”运算，表示“并且”的意思，具有惰性求值特点。以exp1 and exp2为例，如果exp1的值等价于False则返回exp1的值，否则返回exp2的值
not	“逻辑非”运算，对于表达式not x，如果x的值等价于True则返回False，否则返回True

1.4 运算符语法与功能

Python运算符（2）

运算符	功能说明
in、not in is、is not <、<=、>、>=、==、!=	成员测试，表达式x in y的值当且仅当y中包含元素x时才会为True; 同一性测试，测试两个对象是否为同一个对象的引用。如果两个对象是同一个对象的引用，那么它们的内存地址相同; 关系运算符，用于比较大小，作用于集合时表示测试集合的包含关系
|	“按位或”运算符，集合并集
^	“按位异或”运算符，集合对称差集
&	“按位与”运算符，集合交集
<<、>>	左移位、右移位运算符，书写时是两个英文半角的小于号或大于号，不是中文书名号
+ -	算术加法，列表、元组、字符串合并与连接; 算术减法，集合差集
* 、@ 、/ 、//、%	算术乘法，序列重复；矩阵乘法；真除法，结果为实数；整除法，结果为整数；求余数，字符串格式化

1.4 运算符语法与功能

Python运算符（3）

运算符	功能说明
+x -x ~x	正号 负号，相反数 按位求反
**	幂运算，指数可以为小数，例如3**0.5表示计算3的平方根；具有右结合性特点，表达式3**3**3等价于3**(3**3)
[] . ()	下标，切片； 属性访问，成员访问； 函数定义或调用时用来包含参数，修改表达式计算顺序，声明多行代码为一个语句
[]、()、{}	定义列表、元组、字典、集合，列表推导式（或解析式）、生成器表达式、字典推导式、集合推导式

1.4 运算符语法与功能


```python
# 传统方式
sentence = "The quick brown fox jumps over the lazy dog"
words = sentence.split()
if "quick" in words and "lazy" in words:
    print("Both 'quick' and 'lazy' are in the sentence.")

# 使用 := 运算符
if (words := sentence.split()) and ("quick" in words or "lazy" in words):
    print("Either 'quick' or 'lazy' is in the sentence.")

#在这个例子中，words 被赋值为 sentence.split() 的结果，并且这个赋值发生在条件表达式中。
#这可以减少代码的冗余，并使逻辑更加紧凑。
```

```
Both 'quick' and 'lazy' are in the sentence.
Either 'quick' or 'lazy' is in the sentence.
```

1.4 运算符语法与功能


```python
#一个简单的lambda函数，它接收一个参数并返回它的平方。
square = lambda x: x * x
print(square(5))  # 输出 25
```

```
25
```

```python
add = lambda x, y: x + y
print(add(10, 20))  # 输出 30
```

```
30
```

1.5 内置函数语法与功能

Python常用内置函数（1）

函数	功能简要说明
abs(x)	返回数字x的绝对值或复数x的模
all(iterable)	可迭代对象中所有元素都等价于True时返回True，否则返回False，可迭代对象为空时返回True
any(iterable)	可迭代对象iterable中存在等价于True的元素时返回True，否则返回False，可迭代对象为空时返回False
bin(number)	返回整数number的二进制形式的字符串，例如表达式bin(3)的值是'0b11'
bool(x)	参数x的值等价于True时返回True，否则返回False

1.5 内置函数语法与功能-all、any

all() 常用于检查一系列条件是否全部满足。

```python
# 所有元素都为True
print(all([1, 2, 3]))

# 包含一个False元素
print(all([1, 0, 3]))

# 空列表
print(all([]))

# 使用条件表达式
print(all(x > 0 for x in [1, 2, 3]))
```

```
True
False
True
True
```

any() 常用于检查是否存在满足条件的元素。

```python
# 至少有一个元素为True
print(any([0, 1, 0]))

# 所有元素都为False
print(any([0, False]))

# 空列表
print(any([]))

# 使用条件表达式
print(any(x % 2 == 0 for x in [1, 3, 5]))
```

```
True
False
False
False
```

1.5 内置函数语法与功能

Python常用内置函数（2）

函数	功能简要说明
bytes(iterable_of_ints) bytes(string,encoding[, errors]) bytes(bytes_or_buffer) bytes(int)bytes()	创建指定长度的字节串或把其他类型数据转换为字节串，不带参数时创建空字节串。例如： ➢ bytes(5)表示创建包含5个0的字节串b'\x00\x00\x00\x00\x00'， ➢ bytes((97, 98, 99))表示把若干介于[0,255]区间的整数转换为字节串b'abc'， ➢ bytes((97,))可用于把一个介于[0,255]区间的整数97转换为字节串b'a'， ➢ bytes('计算机', 'utf8')使用UTF-8编码格式把字符串‘计算机’转换为字节串b'\xe8\xae\xa1\xe7\xae\x97\xe6\x9c\xba'； ➢ bytes('计算机', 'gbk')使用GBK编码格式把字符串'董付国'转换为字节串b'\xbc\xc6\xcb\xe3\xbb\xfa'
callable(obj)	它用于检查一个对象是否是可调用的。参数obj为可调用对象时返回True，否则返回False，Python中的可调用对象包括函数、lambda表达式、类、类的方法、对象的方法等。

1.5 内置函数语法与功能-callable方法


```
1  def greet():
2      print("Hello, world!")
3
4  print("greet()是否可以调用: ",callable(greet))
5
6  class MyClass:
7      def method(self):
8          print("This is a method.")
9
10 obj = MyClass()
11 print("类中的method()是否可以调用: ",callable(obj.method))
```

```
greet()是否可以调用:  True
类中的method()是否可以调用:  True
```

1.5 内置函数语法与功能

Python常用内置函数（3）

函数	功能简要说明
chr(i)	它接收一个整数（i）作为参数，并返回一个对应的Unicode字符。其中0 <= i <= 0x10ffff。 print(chr(65)) # 输出: A print(chr(97)). # 输出: a print(chr(20013),chr(22269)) # 输出:中 国
dir(obj)	返回一个对象的所有属性和方法名。这些属性和方法名包括定义在对象所在类的直接属性和方法，及从其祖先类继承来的属性和方法。
divmod(x, y)	计算整商和余数，返回元组(x//y, x%y) divmod(5, 2) # 输出: (2, 1)
enumerate(iterable, start=0)	枚举可迭代对象iterable中的元素，返回包含元素形式为(start, iterable[0]), (start+1, iterable[1]), (start+2, iterable[2]), ...的迭代器对象，start为编号起始值，默认为0

1.5 内置函数语法与功能-dir方法


```
# 列出当前模块的所有属性和方法
print(dir())

# 列出math模块的所有属性和方法
import math
print(dir(math))
```

```
['In', 'MyClass', 'Out', '_', '_2', '_3', '__', '___', '__builtin__', '__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__', '_dh', '_i', '_i1', '_i10', '_i11', '_i12', '_i13', '_i14', '_i15', '_i16', '_i17', '_i18', '_i19', '_i2', '_i20', '_i21', '_i22', '_i23', '_i24', '_i25', '_i26', '_i27', '_i28', '_i3', '_i4', '_i5', '_i6', '_i7', '_i8', '_i9', '_ih', '_ii', '_iii', '_oh', 'exit', 'get_ipython', 'greet', 'obj', 'quit']
['__doc__', '__file__', '__loader__', '__name__', '__package__', '__spec__', 'acos', 'acosh', 'asin', 'asinh', 'atan', 'atan2', 'atanh', 'ceil', 'comb', 'copysign', 'cos', 'cosh', 'degrees', 'dist', 'e', 'erf', 'erfc', 'exp', 'expm1', 'fabs', 'factorial', 'floor', 'fmod', 'frexp', 'fsum', 'gamma', 'gcd', 'hypot', 'inf', 'isclose', 'isfinite', 'isinf', 'isnan', 'isqrt', 'lcm', 'ldexp', 'lgamma', 'log', 'log10', 'log1p', 'log2', 'modf', 'nan', 'nextafter', 'perm', 'pi', 'pow', 'prod', 'radians', 'remainder', 'sin', 'sinh', 'sqrt', 'tan', 'tanh', 'tau', 'trunc', 'ulp']
```

```
# 检查字符串对象是否有upper方法
if 'upper' in dir(str):
    print("字符串对象有upper方法")
```

```
字符串对象有upper方法
```

1.5 内置函数语法与功能-enumerate方法

西北民族大學 NORTHWEST MINZU UNIVERSITY

```python
fruits = ['apple', 'banana', 'cherry']

# 从索引0开始枚举
for index, fruit in enumerate(fruits):
    print(f"Index: {index}, Fruit: {fruit}")
```

```
Index: 0, Fruit: apple
Index: 1, Fruit: banana
Index: 2, Fruit: cherry
```

```python
# 从索引1开始枚举
for index, fruit in enumerate(fruits, start=1):
    print(f"Index: {index}, Fruit: {fruit}")
```

```
Index: 2, Fruit: apple
Index: 3, Fruit: banana
Index: 4, Fruit: cherry
```

1.5 内置函数语法与功能

Python常用内置函数（4）

函数	功能简要说明
eval(source,globals=None,locals=None) 示例如下： x = 10 print(eval("x + 5")) # 输出: 15	用于执行一个字符串形式的Python表达式，并返回表达式的值。参数source为要执行的字符串表达式，globals为定义全局命名空间的字典，locals为定义局部命名空间的字典。 使用 eval 需要非常小心，因为它可执行任意代码。尽量避免使用特别是当有更安全、更明确的方法可以达到相同目的时。
filter(function or None, iterable)	用于从可迭代对象（如列表、元组、集合、字典等）中筛选出满足条件的元素。两个参数：一个函数和一个可迭代对象。函数用于检查每个元素是否满足某个条件，返回值为 True 或 False。
float(x=0)	把整数或字符串x**转换为浮点数**，不加参数时返回实数0.0
help(obj)	返回对象obj的帮助信息，例如help(sum)可以查看内置函数sum()的使用说明，help('')可查看字符串对象的使用说明。

1.5 内置函数语法与功能-filter、help


```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
# 定义一个函数，检查数字是否为偶数
def is_even(num):
    return num % 2 == 0

# 使用filter()筛选出列表中的偶数
even_numbers = filter(is_even, numbers)

print(list(even_numbers))
```

```
[2, 4, 6, 8, 10]
```

```
help(math.comb)
```

```
Help on built-in function comb in module math:

comb(n, k, /)
    Number of ways to choose k items from n items without repetition an
d without order.

    Evaluates to n! / (k! * (n - k)!) when k <= n and evaluates
    to zero when k > n.

    Also called the binomial coefficient because it is equivalent
    to the coefficient of k-th term in polynomial expansion of the
    expression (1 + x)**n.

    Raises TypeError if either of the arguments are not integers.
    Raises ValueError if either of the arguments are negative.
```

1.5 内置函数语法与功能

Python常用内置函数（5）

函数	功能简要说明
hex(number)	返回整数的十六进制形式的字符串 hex(65535)　　输出：'0xffff'
input(prompt=None)	输出参数prompt的内容作为提示信息，接收键盘输入的内容，以字符串形式返回。输入时回车表示结束。
int(x)	返回实数x的整数部分，或把字符串x看作base进制数并转换为十进制，base默认为十进制。 int("123"), int(123.456)输出：123
iter(iterable)	获取可迭代对象的迭代器；或者持续调用参数callable指定的可调用对象。
len(obj)	返回可迭代对象obj包含的元素个数，适用于列表、元组、集合、字典、字符串以及range对象。

1.5 内置函数语法与功能-input方法


```
1  # 获取用户的名字
2  name = input("请输入您的名字: ")
3
4  # 获取用户的年龄
5  age = input("请输入您的年龄: ")
6
7  # 打印用户输入的信息
8  print(f"您好, {name}! 您今年{age}岁。")
```

请输入您的名字： 李明

单击回车键，结束输入

请输入您的名字： 李明

请输入您的年龄： 25

单击回车键，结束输入

```
请输入您的名字: 李明
请输入您的年龄: 25
您好, 李明! 您今年25岁。
```

不论用户输入什么内容，input()一律返回字符串（不包含最后的回车键），必要的时候可以使用内置函数int()和float() 对用户输入的内容进行类型转换。

使用input()函数接收列表、元组、字典、集合等类型的数据时，需要使用eval()函数进行转换，不能使用list()、tuple()、dict()、set()。

```
1  lst = eval(input('请输入一个包含若干大于2的自然数的列表: '))
2  print(lst)
3  print(type(lst))
4  print('列表中所有元素之和为: ', sum(lst))
```

```
请输入一个包含若干大于2的自然数的列表: 1,2,3,4,5,6,7,8,9
(1, 2, 3, 4, 5, 6, 7, 8, 9)
<class 'tuple'>
列表中所有元素之和为:  45
```

1.5 内置函数语法与功能-iter方法


```python
# 创建一个字符串
my_string = "Hello"

# 使用iter()获取迭代器
my_iterator = iter(my_string)

# 使用next()获取迭代器的下一个字符
print(next(my_iterator))  # 输出: 'H'
print(next(my_iterator))  # 输出: 'e'

# 使用for循环迭代字符串
for char in my_string:
    print(char)
```

```
H
e
H
e
l
l
o
```

1.5 内置函数语法与功能

Python常用内置函数（6）

函数	功能简要说明
list(iterable=())tuple(iterable=()) dict()、dict(mapping)、 dict(iterable)、dict(**kwargs) **set**()、set(iterable)	把对象iterable转换为列表、元组、字典或集合并返回，不加参数时返回空列表、空元组、空字典、空集合
map(func, *iterables)	用于将一个函数应用于一个或多个可迭代对象中的每个元素，并返回一个迭代器(map对象)。该迭代器会生成该函数应用于每个元素的结果。map() 函数接受两个参数：一个函数和一个或多个可迭代对象。
max(iterable)	返回可迭代对象iterable或多个位置参数中最大的元素
min(iterable)	返回可迭代对象iterable或多个位置参数中最小的元素

1.5 内置函数语法与功能-map方法


```python
numbers1 = [10, 20, 30, 40, 50]
numbers2 = [55, 44, 33, 22]

# 定义一个函数，将两个元素相加
def add(x, y):
    return x + y

# 使用map()函数将两个列表中的元素相加
summed_numbers = map(add, numbers1, numbers2)

print(type(summed_numbers))
# 打印结果
for num in summed_numbers:
    print(num)
```

```
<class 'map'>
65
64
63
62
```

1.5 内置函数语法与功能

Python常用内置函数（7）

函数	功能简要说明
next(iterator[, default])	返回迭代器对象iterator中的下一个元素，如果iterator为空则返回参数default的值，iterable为空且不指定default参数时会抛出StopIteration异常
oct(number, /)	返回整数参数number的八进制形式的字符串， 例，oct(511) 返回 '0o777'
open(file, mode='r', buffering=-1, encoding=None, errors=None, newline=None, closefd=True, opener=None)	以参数mode指定的方式打开参数file指定的文件并返回文件对象。参数file表示要打开或创建的文件路径；操作文本文件时必须使用参数encoding指定正确的编码格式（默认为UTF-8）。
ord(c, /)	返回1个字符c的Unicode编码
print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)	输出函数，可以输出一个或多个值，sep参数表示相邻数据之间的分隔符（默认为空格），end参数用来指定输出完所有值后的结束符（默认为换行符）

1.5 内置函数语法与功能-open方法


```
1  # 打开一个文件进行读取，'r'：读取模式，默认模式。
2  with open('example.txt', 'r') as file:
3      content = file.read()
4      print(content)
5
6  # 打开一个文件进行写入，'w'：写入模式，会覆盖原有文件。
7  with open('example.txt', 'w') as file:
8      file.write('铸牢中华民族共同体意识')
9
10 # 打开一个文件进行追加，'a'：追加模式，会在原有文件末尾添加内容。
11 with open('example.txt', 'a') as file:
12     file.write('\n构建人类命运共同体')
13
```

1.5 内置函数语法与功能-print方法


```python
import datetime

print(1, 2, 3, 4, 5)              # 默认情况，使用空格作为分隔符
print(1, 2, 3, 4, 5, sep=',')     # 指定使用逗号作为分隔符

print(3, 5, 7, end=' ')           # 输出完所有数据之后，以空格结束，不换行
print(9, 11, 13)                  # 在同一行继续输出

width = 20
height = 10
print(f'{width=},{height=},area={width*height}')

today = datetime.date.today()     # 获取今天的日期
print(f'{today.year=}')           # 查看日期所在的年份

data = {'a':97, 'b':98, 'c':99}
print(f'{data["a"]=}')            # 查看字典中指定“键”对应的值
# 在可迭代对象前面加一个星号，表示把其中所有元素取出来作为函数的位置参数，这属于星号表达式的语法。
print(*data.items(), sep=',')

```

```
1 2 3 4 5
1,2,3,4,5
3 5 7 9 11 13
width=20,height=10,area=200
today.year=2024
data["a"]=97
('a', 97),('b', 98),('c', 99)
```

1.5 内置函数语法与功能

Python常用内置函数（8）

函数	功能简要说明
range(stop) **range**(start, stop[, step])	用于生成一个整数序列，返回一个range对象，包含**左闭右开**区间[start, stop)内以step为步长的整数，其中start默认为0，step默认为1 print(list(range(1,10, 2)))　　#输出：[1, 3, 5, 7, 9]
reversed(sequence)	返回可迭代对象中所有元素逆序后组成的迭代器对象。 print(list(reversed([10,20,30,40,50])))　　#输出：[50, 40, 30, 20, 10]
round(number, ndigits=None)	对实数number进行四舍五入，若不指定小数位数ndigits则返回整数，参数ndigits可以为负数。 `1 round(1234,0),round(1234,-1),round(1234,-2)` (1234, 1230, 1200)

1.5 内置函数语法与功能

Python常用内置函数（9）

函数	功能简要说明
sorted(iterable,key=None, reverse=False)	用于对可迭代对象（如列表、元组等）进行排序，并返回一个新的排序列表。参数iterable为被排序的可迭代对象。key为关键字，通常是一个函数或None。这个函数将被应用于每个元素以获取排序键。默认为None，表示直接比较元素本身。参数reverse如果设置为True，则按降序排列；如果设置为False（默认），则按升序排列。
str(object='') **str**(bytes_or_buffer [, encoding[, errors]])	创建字符串对象或者把字节串使用参数encoding指定的编码格式转换为字符串，直接调用str()不加参数时返回空字符串
sum(iterable)	返回可迭代对象中所有元素之和。
type(object)	查看对象类型。
zip(*iterables)	组合多个可迭代对象中对应位置上的元素，返回zip对象，其中每个元素为(seq1[i], seq2[i], ...)形式的元组，最终结果中包含的元素个数取决于参数iterables中所有可迭代对象中最短的那个。

1.5 内置函数语法与功能-zip、sorted


```python
# 创建三个列表
list1 = [1, 2, 3, 4]
list2 = ['a', 'b', 'c']
list3 = [10, 20]

# 使用zip()合并三个列表
zipped_lists = zip(list1, list2, list3)

# 打印合并后的列表
for item in zipped_lists:
    print(item)
```

```
(1, 'a', 10)
(2, 'b', 20)
```

- zip() 函数会根据最短的可迭代对象来决定输出的元组数量。

```python
words = ['banana', 'kiwi', 'cherry','Watermelon','Mango']
print(sorted(words))  # 输出: ['Mango', 'Watermelon', 'banana', 'cherry', 'kiwi']

# 使用 key 参数根据长度对字符串列表进行排序
print(sorted(words, key=len))  # 输出:['kiwi', 'Mango', 'banana', 'cherry', 'Watermelon']
```

1.5 内置函数语法与功能-str方法


```
text = "铸牢中华民族共同体意识"
encoded_bytes = text.encode('utf-8')  # 获取UTF-8编码的字节序列
print(encoded_bytes)
print()
print(list(encoded_bytes))
#print(len(list(encoded_bytes)))
```

```
b'\xe9\x93\xb8\xe7\x89\xa2\xe4\xb8\xad\xe5\x8d\x8e\xe6\xb0\x91\xe6\x97\x8f\xe5\x85\xb1\xe5\
x90\x8c\xe4\xbd\x93\xe6\x84\x8f\xe8\xaf\x86'

[233, 147, 184, 231, 137, 162, 228, 184, 173, 229, 141, 142, 230, 176, 145, 230, 151, 143,
229, 133, 177, 229, 144, 140, 228, 189, 147, 230, 132, 143, 232, 175, 134]
```

```
string_representation = str(encoded_bytes, encoding='utf-8')

# 打印转换后的字符串
print(string_representation)
```

```
铸牢中华民族共同体意识
```

1.5 内置函数语法与功能

dir()、help()

内置函数dir()和help()对于学习和使用Python非常重要，一定要善于使用。其中**dir()函数不带参数时可以列出当前作用域中的所有标识符**，带参数时可以用于查看指定模块或对象中的成员；help()函数带参数时用于查看对象的帮助文档，不带参数时进入交互式帮助环境，按字母q退出,(类似于input命令执行后的交互方式)。

```
1 import math
2 print(dir())
```

```
['In', 'Out', '_', '_19', '_20', '_21', '__', '___', '__builtin__', '__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__', '_dh', '_i', '_i1', '_i10', '_i11', '_i12', '_i13', '_i14', '_i15', '_i16', '_i17', '_i18', '_i19', '_i2', '_i20', '_i21', '_i22', '_i3', '_i4', '_i5', '_i6', '_i7', '_i8', '_i9', '_ih', '_ii', '_iii', '_oh', 'data', 'datetime', 'exit', 'get_ipython', 'height', 'lst', 'math', 'quit', 'today', 'width', 'words']
```

```
1 help()
```

```
Welcome to Python 3.9's help utility!

If this is your first time using Python, you should definitely check out
the tutorial on the Internet at https://docs.python.org/3.9/tutorial/.

Enter the name of any module, keyword, or topic to get help on writing
Python programs and using Python modules.  To quit this help utility and
return to the interpreter, just type "quit".

To get a list of available modules, keywords, symbols, or topics, type
"modules", "keywords", "symbols", or "topics".  Each module also comes
with a one-line summary of what it does; to list the modules whose name
or summary contain a given string such as "spam", type "modules spam".

help>
```

1.5 内置函数语法与功能-dir、help


```
1 import secrets
2 rndGenerater = secrets.SystemRandom() # 创建对象，用来生成安全随机数
3 print("查看对象方法列表")
4 print(dir(rndGenerater))
5 print("查看对象方法的帮助文档")
6 help(rndGenerater.choices)
```

```
查看对象方法列表
['VERSION', '__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__form
at__', '__ge__', '__getattribute__', '__getstate__', '__gt__', '__hash__', '__init__', '__i
nit_subclass__', '__le__', '__lt__', '__module__', '__ne__', '__new__', '__reduce__', '__re
duce_ex__', '__repr__', '__setattr__', '__setstate__', '__sizeof__', '__str__', '__subclass
hook__', '__weakref__', '_notimplemented', '_randbelow', '_randbelow_with_getrandbits', '_r
andbelow_without_getrandbits', 'betavariate', 'choice', 'choices', 'expovariate', 'gammavar
iate', 'gauss', 'gauss_next', 'getrandbits', 'getstate', 'lognormvariate', 'normalvariate',
'paretovariate', 'randbytes', 'randint', 'random', 'randrange', 'sample', 'seed', 'setstate
', 'shuffle', 'triangular', 'uniform', 'vonmisesvariate', 'weibullvariate']
查看对象方法的帮助文档
Help on method choices in module random:

choices(population, weights=None, *, cum_weights=None, k=1) method of random.SystemRandom i
nstance
    Return a k sized list of population elements chosen with replacement.

    If the relative weights or cumulative weights are not specified,
    the selections are made with equal probability.
```

1.5 内置函数语法与功能


```python
import secrets

# 创建一个可迭代对象，例如一个列表
options = ['apple', 'banana', 'cherry', 'orange', 'kiwi']

# 使用secrets.choice()函数随机选择一个元素
random_choice = secrets.choice(options)

# 打印随机选择的元素
print(random_choice)
```

```
kiwi
```